{EPITECH}

TEMPLATE REPORT

REVERSE BINARY

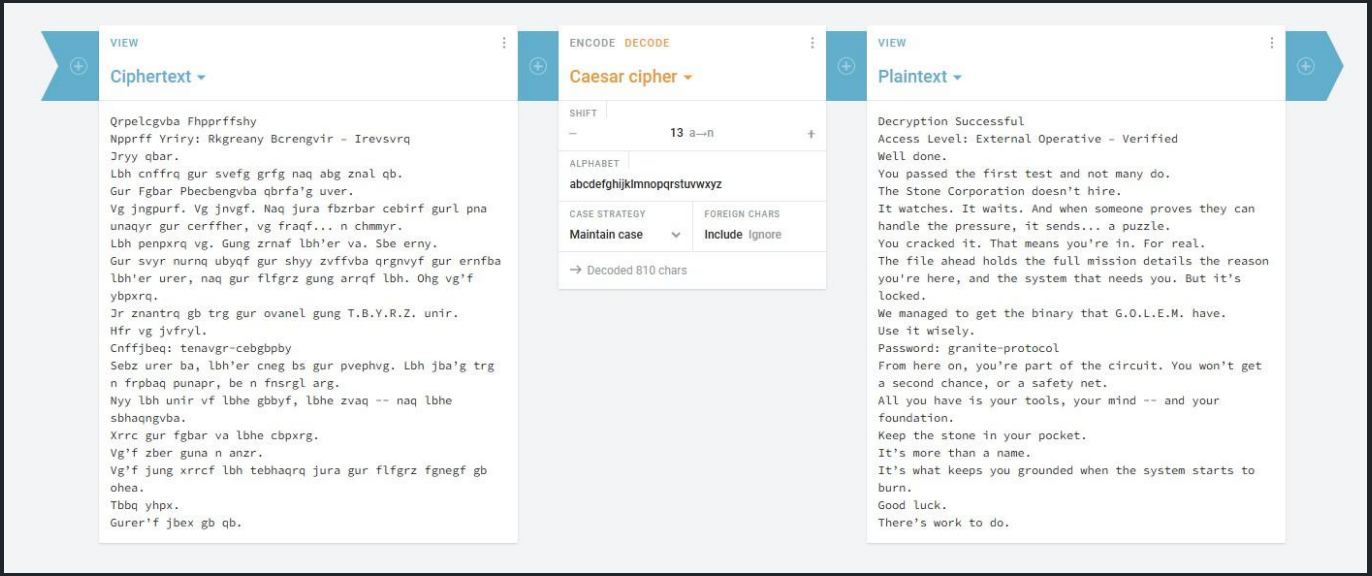


REPORT

Black-box Audit

In this phase, we analyzed the binary using reverse engineering tools and manual fuzzing.

List of the tools you used:
- strings
- upx
- caesar-cipher (url: https://cryptii.com/pipes/caesar-cipher/)



-

Key discoveries:
- identified that the binary was packed with UPX.
- discovered credentials (password: admin123), enabling unauthorized acces to protected commands
- found that the application leaks data.
- found insecure input handling, identifying a format string vulnerability and a logic flaw regarding negative input values.

White-box Audit

With access to source code, we performed a full code review and vulnerability assessment.

Focus area:

- Input validation
- Memory safety
- Privilege escalation paths

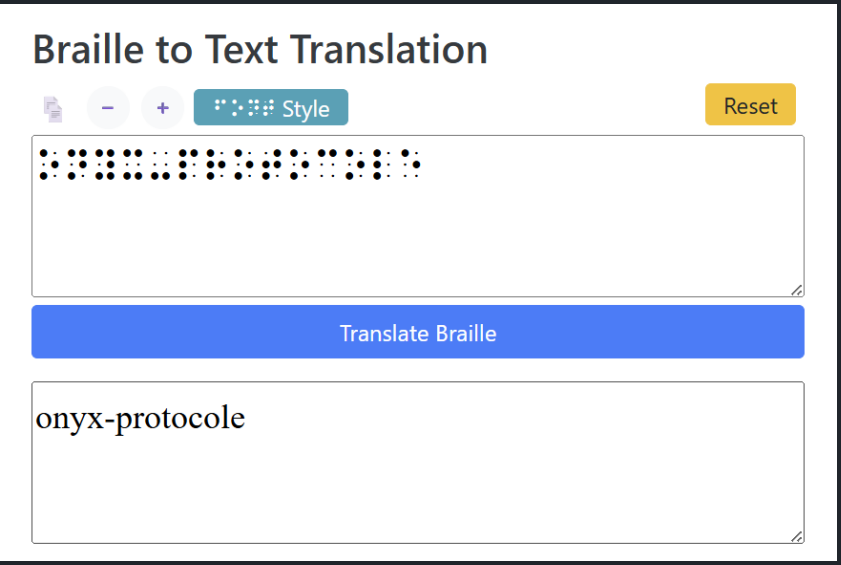
List of the tools you used:

- Strings

- C code

Final report

In order to gain access to the last pdf (Obsidian_final.pdf), we decrypted the password, which was encoded using the Braille alphabet, using the following website: https://abcbraille.com/braille.



Vulnerability Report

Below is the list of vulnerabilities discovered:

Vulnerability #1: System compromise

Severity: Critical
Type: Privilege Escalation / Bypass
Location: activate_emergency_protocols
Discovered in: Black-box

Description:
The application hid sensitive information using a binary packing (UPX). Using the utilitary upx, this was easily bypassed. Once the binary was unpacked, the application's internal strings revealed multiple passwords and flags used to bypass authentication for "Emergency protocols".

Proof of concept:

- detection of packing (analysis with strings failed to show readable logic, revealing a header)

```
5591    _j<X
5592    $Info: This file is packed with the UPX executable packer http://upx.sf.net $
5593    $Id: UPX 4.22 Copyright (C) 1996-2024 the UPX Team. All Rights Reserved. $
```

- filter with sensitive keywords

```
diana@Diana:~/cybersecurity/binaryV4$ strings obsidian | grep -iE "pass|admin"
Enter emergency password:
No password entered, emergency protocols not activated.
admin123
{Emergency protocols activated, you are now admin !}
- activate_emergency_protocols: Activate emergency protocols (requires admin).
Secret mode unlocked! Welcome, admin.
{ADMIN4242}
ThisIsTheBestPassword
{Correct password! Welcome, admin.}
Please enter your password:
```

- exploitation

```
obsidian> activate_emergency_protocols
Enter emergency password: admin123
{Emergency protocols activated, you are now admin !}
obsidian>
```

Impact:
An attacker can gain admin privileges over the application, managing to bypass all security layers, granting unauthorized access to multiple functions.

Fix Summary:
- We removed the hardcoded (static) password “admin123”, which was saved in the .rodata section of the executable.
- We now retrieve it dynamically using an environment variable. So, the program no longer knows the password and only fetches it at runtime (when the function runs), rather than when the code is compiled (compile time).
- Now, when the attacker uses strings obsidian, they will only see the string “ADMIN_PASSWORD”.
- Note that after compiling the new code, we have to write export ADMIN_PASSWORD=admin123 (for bash) or setenv ADMIN_PASSWORD admin123 (for tcsh) into the terminal in order to setup the authentication system.
- This ensures that sensitive information is no longer compiled into the binary, thus preventing attackers from using tools like upx or strings to bypass security layers and gain unauthorized access.
- Patch file: fixes/activate_emergency_protocols.c.patch

Vulnerability #2: Unauthorized Access to Functions

Severity: Critical
Type: Priviledge Escalation / Bypass
Location: Secret functions
Discovered in: Black-box

Description:
While exploring the program's code with the strings tool, we discovered two hidden commands: unlock_secret_mode and trigger_emergency_shutdown. Initially, the application denied execution of these commands, displaying error messages which state that we didn't have the "required priviledges". However, by leveraging exploit #1, we escalated our status to admin and was able to successfully execute these hidden commands.

Proof of Concept:

```
2917    run_diagnostic
2918    enable_remote_access
2919    quit
2920    help
2921    load_fuel_rods
2922    log_system_event
2923    unlock_secret_mode
2924    load_config
2925    history
2926    configure_cooling_system
2927    trigger_emergency_shutdown
2928    init_labo
2929    init_workers
2930    call_api
```

```
obsidian> unlock_secret_mode
Access denied. You do not have the required privileges.
obsidian> trigger_emergency_shutdown
You are not authorized to trigger an emergency shutdown.
obsidian> activate_emergency_protocols
Enter emergency password: admin123
{Emergency protocols activated, you are now admin !}
obsidian> unlock_secret_mode
Secret mode unlocked! Welcome, admin.
{ADMIN4242}
obsidian> trigger_emergency_shutdown
{SHUTDOWN}diana@Diana:~/cybersecurity/binaryV4$
```

Impact:
Being able to gain admin access can lead to system compromise.

Fix Summary:

- Both unlock_secret_mode and trigger_emergency_shutdown are safely written by checking for the global IS_ADMIN variable before granting access to the flags.
- The unauthorized execution shown in the screenshot above is a chained exploit made possible by the Privilege Escalation vulnerability in the activate_emergency_protocols (#1).
- Because we have secured the authentication method in the exploit before this one, the functions are not exposed anymore.

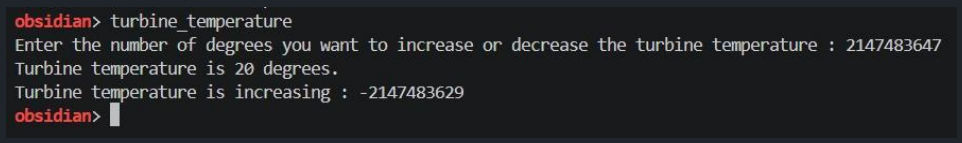
Vulnerability #3: Integer Overflow

Severity: Critical
Type: Integer Overflow; Improper Input Validation
Location: obsidian_lib.so, turbine_temperature
Discovered in: Black-box

Description:
The function does not validate the upper bounds of the temperature change input. The vulnerability comes from the use of a 32-bit signed integer variable to store the temperature. In C, a signed 32-bit integer has a maximum value of $2^{31} - 1$, which is 2.147.483.647. The program adds the provided number to the current temperature (20°C), thus exceeding the storage capacity of the variable. The most significant bit (the sign bit) is flipped and the number is interpreted as negative. The solution is to implement a check for a maximum input threshold before performing the sum.

Proof of Concept:

1. Run ./obisidian
2. Execute init_steam_turbine
3. Execute turbine_temperature
4. Input 2147483647



Impact:
- Logic manipulation: unintended negative values can cause unwanted events that affect the operation (in this case the way the turbines work will be altered).
- Safety checks: these kinds of systems usually have a threshold for high temperatures that cause the equipment to stop. An attacker could keep the system running at extremely high temperatures, while the program registers them as negative degrees and preventing the automatic emergency shutdown for that kind of temperature

Fix Summary:
- The proposed solution would be to implement strict upper bound validation on the user input before performing operations with the current temperature.
- We do this by adding a threshold check to ensure that the sum of the input and the base temperature does not exceed the maximum capacity of a 32-bit signed integer (2147483647). This way the input is safely rejected and the overflow is avoided.

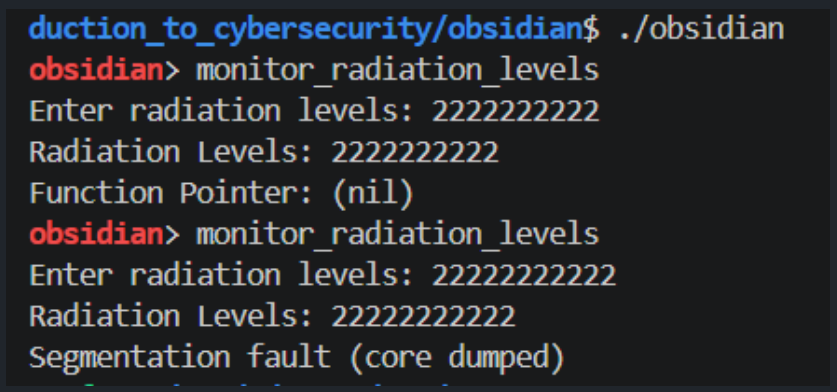
Vulnerability #4: Buffer Overflow

Severity: Critical
Type: Stack-based Buffer Overflow; Function Pointer Hijacking
Location: obsidian, monitor_radiation_levels
Discovered in: Black-box

Description:
The function does not validate user input size before storing it in memory. Providing a string longer than 10 characters causes a segmentation fault. The function allocates a buffer on the stack for this radiation level string and, immediately after, there is a function pointer used by the system. By inputting 11 characters, it overflows its boundaries by overwriting into the first byte of the function pointer.

Proof of Concept:

1. Run ./obsidian
2. Execute monitor_radiation_levels
3. Enter a 11-character number for radiation levels



*Note that the first example in the screenshot is for a 10-character string, while the second one is for an 11-character string

Impact:

- Control Flow Hijacking: an attacker can replace the pointer with the memory address of a malicious function or one that requires admin privileges.

Fix Summary:

- We replaced the gets function with fgets, which takes into account the actual maximum read size based on the buffer's capacity.
- We increased the buffer size to 32 bytes to handle inputs better.
- We also used strcspn to remove the newline character at the end captured by fgets.
- Patch file: fixes/monitor_radiation_levels.c.patch

Vulnerability #5: Sensitive Information Disclosure

Severity: High
Type: Information Disclosure
Location: log_system_event
Discovered in: White-box

Description:
The log_system_event function contains a logic flaw where sensitive information is written to an accessible log file if a specific string is present in the user input. The application uses strstr(input, "leak") to monitor incoming commands. If a user provides an input containing the word leak, the program displays sensitive information in the Data/system.log file.

Proof of Concept:

```
obsidian> log_system_event
Enter command: leak
Logging event: leak
```

log_system_events.c | system.log

Obsidian > Data > system.log

```
1   EVENT: leak
2
3   SECRET_KEY_LEAKED: {SECRET_LOG_12PIERRE34}
```

Impact:
By revealing the exact name and location of the log files, the application provides the attacker with essential reconnaissance data. An attacker could attempt to overwrite system logs, making the application unsafe. Moreover, if an attacker does something bad in the system, they can simply delete or modify the file, making it impossible for an admin to see what happened.

Fix Summary:
- We fixed the information disclosure by adding a strict check for admin privileges before inputting the problematic string.
- We included the obsidian.h file and added a check for IS_ADMIN.
- Patch file: fixes/ log_system_events.c.patch

Vulnerability #6: Insecure Log Handling

Severity: High
Type: Log injection
Location: read_turbine_config
Discovered in: Black-box

Description:
The application leaks path information regarding the folder. By leveraging the path leakage identified in vulnerability #3, an attacker knows the application writes to and reads from Data/system.log. The attacker can manually insert keywords into system.log, tricking the application into executing those strings as internal events if the log file is loaded as a config file.

Proof of Concept:

```
obsidian> read_turbine_config
Enter the configuration file name: d
Error: Unable to open file: Data/d
obsidian> read_turbine_config
Enter the configuration file name: system.log
Reading configuration file: Data/system.log
EVENT: ls

obsidian>
```

```
obsidian> read_turbine_config
Enter the configuration file name: system.log
Reading configuration file: Data/system.log
EVENT: ls

obsidian> quit
Exiting program...
diana@Diana:~/cybersecurity/binaryV4$ echo "EVENT: ls -l" >> Data/system.log
diana@Diana:~/cybersecurity/binaryV4$ ./obsidian
obsidian> read_turbine_config
Enter the configuration file name: system.log
Reading configuration file: Data/system.log
EVENT: ls

EVENT: ls -l
obsidian>
```

Impact:
This allows an attacker to manipulate the application's logic. By manipulating the system.log file that the program later uses, an attacker can maybe later trigger unintended events.

Vulnerability #7: Boundary Logic Flaw

Severity: Medium
Type: Integer Overflow / Improper Input Validation
Location: obsidian, set_reactor_power
Discovered in: White-box

Description:
The function contains a critical vulnerability regarding the handling of signed integers. The program uses a 32-bit signed integer for the power level. The program contains an explicit check that, instead of rejecting dangerous input, triggers an emergency shutdown if the addition exceeds the maximum capacity of the integer.

Proof of Concept:

1. Run ./obsidian
2. Execute set_reactor_power
3. Enter 2147483647 (maximum value for a signed 32-bit int)

```
obsidian> set_reactor_power
Enter reactor power level: 2147483647

Reactor power adjustment may be incorrect.
Reactor systems are behaving erratically!
Reactor core temperature rising uncontrollably...
{12EXPLOSION34}
Emergency shutdown initiated!
```

Impact:
- Privilege escalation: an attacker could possibly trigger a dangerous state, such as the meltdown.
- Denial of service: after the meltdown is initiated, the system becomes unusable.

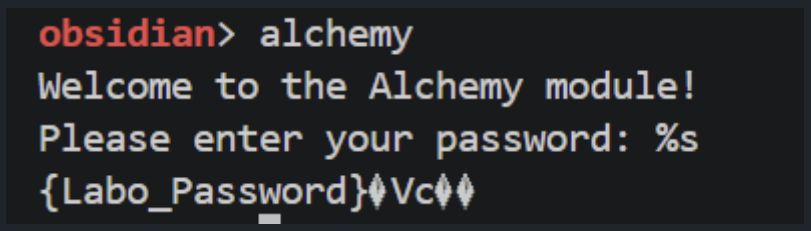
Fix Summary:
- We replaced the atoi function with strtol in order to convert and validate the input before turning it into a 32-bit integer.
- We updated the conditional logic to reject negative values and integers that are too large as to avoid the integer overflow.
- We used return instead of exit(1) to go back to the prompt.
- We increased the buffer size from 11 to 32 bytes to handle input better.
- Patch file: fixes/set_reactor_power.c.patch

Vulnerability #8: Uncontrolled Format String

Severity: High
Type: Format string
Location: alchemy command
Discovered in: Black-box

Description:
The vulnerability exists in the way the alchemy function handles user input for the password prompt. Specifically, the application passes user-supplied data directly to an output function (such as printf()) without a format string. This allows an attacker to write format specifiers, which the application then interprets as instructions to read from the stack.

Proof of Concept:



Impact:
The exploitation of this format string vulnerability can lead to critical security compromise, such as information disclosure. An attacker can use specifiers like "%s" to leak sensitive data from the stack.

Fix summary:
Since the source code for the alchemy command is unavailable, this vulnerability cannot be fixed directly in the binary. However, the fix for this issue requires updating the code to ensure that user input is never passed as the first argument of an output function. Instead of printf(user_input), it should be printf("%s", user_input).

Vulnerability #9: Improper Handling of Negative Input

Severity: High
Type: Improper Input Validation
Location: run_turbine command
Discovered in: Black-box

Description:
The run_turbine command contains a logic flaw in its input validation. The application explicitly states that it only accepts values between 0 and 15. For a higher value, the program displays a proper error message. However, the application fails to properly handle negative values. For the value -1, the check (which is likely if value <= 15) is bypassed and the program enters an infinite loop.

Proof of Concept:

```
obsidian> run_turbine
Enter the number of rotation that the turbine will do (between 0 and 15): 16
Invalid number of rotations.
```

```
obsidian> run_turbine
Enter the number of rotation that the turbine will do (between 0 and 15): -1
Turbine is running... 1/-1
Turbine is running... 2/-1
Turbine is running... 3/-1
Turbine is running... 4/-1
Turbine is running... 5/-1
Turbine is running... 6/-1
Turbine is running... 7/-1
Turbine is running... 8/-1
Turbine is running... 9/-1
Turbine is running... 10/-1
Turbine is running... 11/-1
Turbine is running... 12/-1
Turbine is running... 13/-1
Turbine is running... 14/-1
Turbine is running... 15/-1
Turbine is running... 16/-1
{ERR0R TURB1NE CAN'T ST0P}
Turbine is running... 17/-1
{ERR0R TURB1NE CAN'T ST0P}
```

Impact:
The main impact is that an attacker can crash the program or make it freeze by forcing it into an infinite loop. Since the program is busy running the turbine forever, it cannot process other commands. Also, this shows that the program has improper input validation, meaning we can bypass the rules set by the developers.

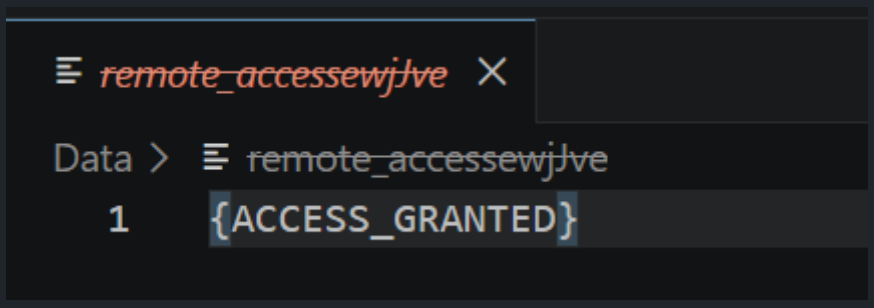
Fix summary:
Since the source code for the run_turbine command is unavailable, this input validation vulnerability cannot be fixed using only the binary. However, we recommend implementing an additional input validation. This check should block any negative numbers, instead of passing them to the program.

Vulnerability #10: Insecure Temporary File Storage

Severity: Medium
Type: Information Disclosure
Location: turbine_remote_access
Discovered in: Black-box

Description:
When the turbine_remote_access runs, the program creates a temporary file in the Data/ folder. Although it is a temporary file and the application deletes it after the command ends, the file is still viewable while the function runs.

Proof of Concept:



Impact:
The impact is information disclosure. Any sensitive information stored in temporary files is easy to access.

Fix summary:
Since we don't have the source code for the turbine_remote_access command, we fixed the vulnerability by hardening the folder permissions. Using the "chmod 700" command, we restricted the directory access. This ensures that while the binary can still create and delete its temporary files during execution, any other user, besides the owner, is blocked from reading the contents of the Data/ folder.

Vulnerability #11: Sensitive Data Leakage via Type Confusion

Severity: Medium
Type: Improper Access Control / Type Confusion
Location: run_diagnostic command
Discovered in: White-box

Description:
The run_diagnostic command provides an integrated diagnostic interface. While the advanced mode operates correctly by assigning an integer address to the integer pointer (data = &diagnostic_code), the debug mode introduces a Type Confusion vulnerability. In this mode, the program forces the memory address of a character array (sensitive_data) into the same data pointer. Because the application fails to implement authorization checks and re-casts the pointer back to a string during output, it leaks hardcoded information when debug keyword is typed.

Proof of Concept:

```
obsidian> run_diagnostic
Enter diagnostic mode (normal/debug/advanced): debug
Running diagnostic...
Diagnostic result: {SECRET DIAGNOSTIC KEY}
Performing system health check...
System health: OK
Diagnostic_complete.
```

Impact:
The impact is information disclosure. Unauthorized users can extract internal data by exploiting the type mismatch.

Fix summary:
The vulnerability was resolved by removing the unsafe type mismatch and implementing role-based access. The type confusion was fixed by assigning an integer value (1001) to the data pointer for the debug mode. As for the output logic, the application now checks the user's priviledge before displaying the sensitive_info string.
- Patch file: fixes/run_diagnostic.patch

Vulnerability #12: UAF leading to Data Exposure

Severity: High
Type: Use-After-Free (UAF)
Location: check_cooling_pressure command
Discovered in: White-box

Description:
The check_cooling_pressure function suffers from a Use-After-Free vulnerability. The pointer data, which initially contains the string "Pressure OK", is released with free(). Because the application does not nullify the pointer, it creates a dangling pointer that the application uses to perform logic checks. When the program reaches if (strcmp(data, "Pressure OK")) statement, it performs a comparison using overwritten memory. Since the memory at the data address is corrupted after being freed, the comparison with "Pressure OK" fails and return non-zero. Since any non-zero is considered true, the application displays sensitive information.

```
free(data);
sleep(3);
if (strcmp(data, "Pressure OK")) {
    printf("Sensitive Info: %s\n", sensitive_info);
}
```

Proof of Concept:

```
obsidian> check_cooling_pressure
Cooling pressure check in progress...
Simulating sensor reading...
Sensor reading complete.
Checking cooling pressure...
Cooling pressure status: Pressure OK
Sensitive Info: {Sensitive Data}
Temporary buffer: Temporary data: 15
Simulating sensor reading...
Sensor reading complete.
Cooling pressure check complete.
```

Impact:
The impact is information disclosure, by allowing unauthorized access to sensitive data. Moreover, accessing freed memory is undefined behavior which may lead to crashes or further memory corruption if the pointer is used for write operations.

Fix summary:
The vulnerability was fixed by setting the data pointer to NULL after deallocation. Moreover, the conditional logic was updated to check for NULL status to prevent a Segmentation Fault, caused by the strcmp function.
- Patch file: fixes/check_cooling_pressure.patch

Vulnerability #13: Sensitive Data Exposure

Severity: Medium
Type: Information Disclosure
Location: simulate_meltdown command
Discovered in: White-box

Description:
The simulate_meltdown function contains a logical flow that leaks sensitive information when the system enters a meltdown state. The vulnerability resides in the error-handling logic: when a random_number variable is smaller than 5, the application triggers a "Critical Error" message. The program prints the hardcoded secret_code directly to the standard output.

Proof of Concept:

```
obsidian> simulate_meltdown
Generated random number: 23
Alert: Reactor core temperature stable.
Reactor core temperature: 23
Reactor core status: Reactor Stable
obsidian> simulate_meltdown
Generated random number: 67
Reactor core temperature normal.
Reactor core temperature: 67
Reactor core status: Reactor Normal
obsidian> simulate_meltdown
Generated random number: 35
Alert: Reactor core temperature stable.
Reactor core temperature: 35
Reactor core status: Reactor Stable
obsidian> simulate_meltdown
Generated random number: 29
Alert: Reactor core temperature stable.
Reactor core temperature: 29
Reactor core status: Reactor Stable
obsidian> simulate_meltdown
Generated random number: 2
Meltdown simulated! Reactor core is overheating.
Critical Error: Secret Code Leaked: {MELTDOWN1234}
```

Impact:
The data exposure provides an attacker with a credential which may be used in order to bypass future security checks.

Fix summary:
Fixed an Information Disclosure vulnerability. The vulnerability was resolved by removing the sensitive data reference from the printf statement and replacing it with a generic error message.
- Patch file: fixes/simulate_meltdown.patch

Vulnerability #14: Logical Gap in Input Validation

Severity: Medium
Type: Information Disclosure
Location: load_fuel_rods command
Discovered in: White-box

Description:
The load_fuel_rods function has an incomplete conditional flaw. The input validation uses an if / else if statement that explicitly handles cases where the input is greater than 10 or between 1 and 9. However, it fails to account for the values of 0, 10 and negative integers. When the user input is 0/10/-2, the program bypasses the if / else if statements and displays hardcoded sensitive code.

Proof of Concept:

```
obsidian> load_fuel_rods
Loading fuel rods...
Enter the number of fuel rods to load (max 10): 0

Sensitive Data:
Secret Key: {The secret stone is here !}]r]0
```

```
obsidian> load_fuel_rods
Loading fuel rods...
Enter the number of fuel rods to load (max 10): 10

Sensitive Data:
Secret Key: {The secret stone is here !}]r]10
```

```
obsidian> load_fuel_rods
Loading fuel rods...
Enter the number of fuel rods to load (max 10): -2

Sensitive Data:
Secret Key: {The secret stone is here !}#-2
```

Impact:
The data exposure provides an attacker with a credential which may be used in order to bypass future security checks. By identifying gaps in the program's logic, an attacker can access sensitive code that was indented to be protected.

Fix summary:
Fixed the input validation logic by strictly accepting values between 0 and 10, rejecting negative numbers, as well as numbers higher than 10.
- Patch File: fixes/load_fuel_rods.patch

Vulnerability #15: Uncontrolled resource consumption

Severity: High
Type: Resource Exhaustion
Location: init_workers command
Discovered in: White-box

Description:
The init_workers command contains a vulnerability where it does not properly manage or limit the generation of system resources. When the command is called, the application starts workers processes, but it lacks an upper bound or safety check to stop them. Because of this missing boundary, an attacker can force the binary into a loop that continuously requests resources from the system without ever releasing them.

Proof of Concept:
1. Run init_workers
2. Input the number of workers

```
Initializing worker 1...
Fork failed: Resource temporarily unavailable
Worker 14 is ready.
Initializing 90 workers...
Initializing worker 1...
Fork failed: Resource temporarily unavailable
Worker 8 is ready.
Initializing 90 workers...
Initializing worker 1...
Fork failed: Resource temporarily unavailable
Worker 5 is ready.
Initializing 90 workers...
Initializing worker 1...
Fork failed: Resource temporarily unavailable
```

Impact:
The impact is a complete Denial of Service due to Resource Exhaustion. Since the binary doesn't stop creating processes, it leaves zero resources to the rest of the system, causing the machine to become frozen, unresponsive to any other command.

Fix summary:
Since the source code for this function is unavailable, this vulnerability cannot be fixed using only the binary. However, to prevent this issue from crashing the machine, a limit should be implemented, so we can restrict the maximum number of child processes.

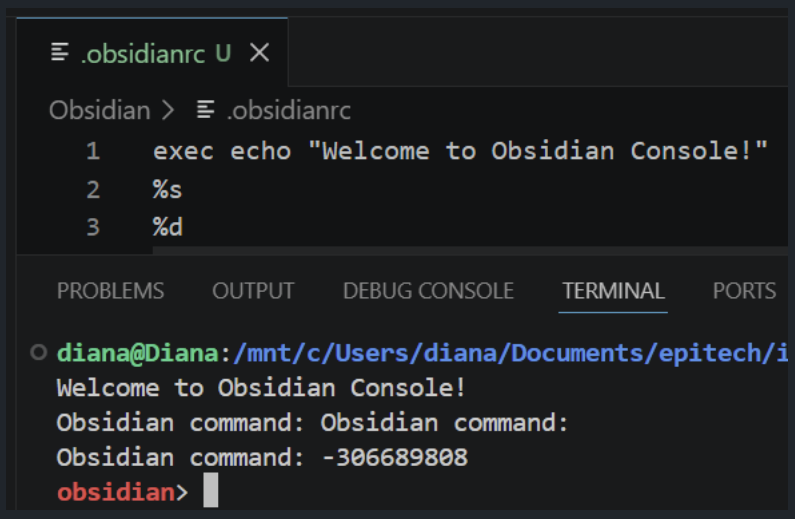
Vulnerability #16: Format string in .obsidianrc loading

Severity: High
Type: Format string
Location: src/utils.c (load_obsidianrc function)
Discovered in: White-box

Description:
During the white-box analysis, a Format String vulnerability was identified in the load_obsidianrc function at line 44. The application uses the statement printf(line) to output data read from the .obsidianrc file. Because the line variable is passed directly as the first argument to printf() without any explicit format specifier, such as "%s", the function treats any % characters inside the file as directives rather than literal text. This allows the function to pass information from the program's stack.

Proof of concept:

```
src/utils.c: In function 'load_obsidianrc':
src/utils.c:44:18: warning: format not a string literal and no format arguments [-Wformat-security]
   44 |             printf(line);
      |                    ^~~~
```



Impact:
The impact of this vulnerability is Information Disclosure. By supplying format specifiers, an attacker can leak sensitive data residing on the stack.

Fix summary:
The fix for this issue requires updating the code to ensure that user input is never passed as the first argument of an output function. Instead of printf(line), it should be printf("%s", line).

Vulnerability #17: Makefile

Severity: High
Type: Memory corruption
Location: Makefile
Discovered in: White-box

Description:

The Makefile contains a series of compilation flags that disable some security mechanisms. This could result in the binary becoming susceptible to memory corruption exploits, which include stack-based overflows and arbitrary code execution.

- -fno-stack-protector -> disables stack canary protections; canaries are randomly generated values placed on the stack to detect buffer overflows before the instruction pointer is modified. Without them, an attacker could overwrite return addresses stored on the stack without triggering an alert.
- -fno-pie -> disables Position Independent Executable (PIE); this flag prevents the binary from being loaded at random memory addresses. When the binary is loaded at a fixed memory address, the attackers could predict the location of the code.
- -Wno-implicit-function-declarations -> suppresses warnings; the flag forces the compiler to ignore instances where functions are called before being declared. Hiding these warnings can lead to malicious function execution or undefined behavior at runtime.

Proof of concept:

```
CFLAGS += -Wall -Wextra -fPIC -fno-stack-protector -Wno-implicit-function-declaration -fno-pie
```

Impact:

- These flags create an exploitable environment. An attacker could overflow the buffer to overwrite adjacent memory or redirect the execution flow to their own malicious payload, and the memory location and layout could become predictable.

Fix Summary:

- To secure the binary, the removal of these flags is necessary.
- CFLAGS += -Wall -Wextra -fPIC

v 1.0

